FICHE APPRENANT · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 2

La base de données MySQL

Durée : 5 jours SQL Migrations Eloquent Relations Enums PHP

Objectifs de la semaine

- écrire des requêtes SQL simples (SELECT, WHERE, ORDER BY, JOIN) ;
- traduire un modèle de données en **migrations** Laravel ;
- comprendre les **clés étrangères** et ce qui se passe à la suppression ;
- créer des **modèles Eloquent** et leurs relations ;
- remplir la base avec des données réalistes (**factories** et **seeders**).

1. Les migrations : le schéma de la base en code

Une migration décrit une modification de la base. Elle est versionnée avec Git : toute l'équipe a la même structure.

```
php artisan make:migration create_sites_table
```

```
Schema::create('sites', function (Blueprint $table) {
    $table->id();                              // BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY
    $table->string('code', 20)->unique();      // VARCHAR(20) + index UNIQUE
    $table->string('nom');
    $table->string('ville');
    $table->string('type');
    $table->string('adresse')->nullable();     // peut être NULL
    $table->timestamps();                      // created_at, updated_at
});
```

```
php artisan migrate            # applique les migrations en attente
php artisan migrate:rollback   # annule le dernier lot
php artisan migrate:fresh      # supprime tout et recrée (DEV uniquement !)
```

Exercice — Après `php artisan migrate`, tape `SHOW CREATE TABLE sites;` dans MySQL. Retrouve chaque ligne de la migration dans le SQL généré.

2. Les clés étrangères

```
$table->foreignId('site_id')->constrained()->restrictOnDelete();
```

Cette ligne crée une colonne `site_id` **et** une contrainte : MySQL refusera un `site_id` qui n'existe pas dans `sites`. Le comportement à la suppression du parent est un **choix métier** :

Méthode	Effet quand on supprime le parent	Exemple dans le projet
`restrictOnDelete()`	Suppression **refusée**	Un site qui a des équipements
`nullOnDelete()`	La colonne passe à `NULL`	Un équipement supprimé → l'incident garde son historique
`cascadeOnDelete()`	Les enfants sont **supprimés aussi**	Un incident supprimé → ses suivis disparaissent

Pour une liaison entre deux sites, on nomme les colonnes nous-mêmes :

```
$table->foreignId('site_a_id')->constrained('sites');
$table->foreignId('site_b_id')->constrained('sites');
```

3. Les index

Un index accélère les recherches, comme l'index d'un livre. On indexe les colonnes **souvent filtrées ou triées** : `statut`, `severite`, `debut_at`.

```
$table->string('statut')->default('ouvert')->index();
```

4. Les modèles Eloquent et les relations

```
#[Fillable(['code', 'nom', 'ville', 'type', 'adresse'])]
class Site extends Model
{
    public function equipements(): HasMany
    {
        return $this->hasMany(Equipement::class);
    }
}

class Equipement extends Model
{
    public function site(): BelongsTo
    {
        return $this->belongsTo(Site::class);
    }
}
```

- `#[Fillable]` liste les colonnes qu'on a le droit de remplir en masse (`Site::create($donnees)`). C'est une **protection** : un utilisateur malveillant ne peut pas injecter une colonne non prévue.
- `$site->equipements` renvoie la collection des équipements ; `$equipement->site` renvoie le site.

Quand la clé étrangère n'a pas le nom standard, on la précise :

```
public function siteA(): BelongsTo
{
    return $this->belongsTo(Site::class, 'site_a_id');
}
```

5. Les enums PHP

Plutôt que des chaînes « magiques » éparpillées dans le code, on définit une liste fermée de valeurs :

```
enum Severite: string
{
    case Critique = 'critique';
    case Majeure = 'majeure';
    case Mineure = 'mineure';

    public function label(): string
    {
        return match ($this) {
            self::Critique => 'P1 – Critique',
            self::Majeure => 'P2 – Majeure',
            self::Mineure => 'P3 – Mineure',
        };
    }
}
```

Dans le modèle, un **cast** transforme automatiquement la valeur de la base en enum :

```
protected function casts(): array
{
    return ['severite' => Severite::class, 'debut_at' => 'datetime'];
}
```

`$incident->severite->label()` affiche alors « P1 - Critique ».

6. Factories, seeders et Tinker

- Une **factory** fabrique des modèles avec des données aléatoires (pour les tests).
- Un **seeder** remplit la base avec un jeu de données de démonstration.

```
php artisan migrate:fresh --seed
php artisan tinker
```

```
Incident::with('site')->first()->site->nom;
Site::withCount('equipements')->get();
Liaison::first()->siteB->ville;
```

Exercices MySQL (après le seed)

Écris et exécute ces requêtes **en SQL** dans le client `mysql`, puis retrouve l'équivalent Eloquent.

1. Lister tous les sites triés par ville.
2. Lister les équipements Cisco, avec leur hostname et leur IP.
3. Compter le nombre d'équipements par type.
4. Afficher chaque équipement avec le **nom de son site** (indice : `JOIN`).

5. Afficher chaque liaison avec le code du site A et le code du site B (indice : deux `JOIN` sur la même table, avec des alias).
6. Essayer de supprimer un site qui a des équipements : quel message MySQL renvoie-t-il ? Pourquoi ?

Travail demandé

1. Ajouter les colonnes `role` et `telephone` à la migration `users`.
2. Créer les migrations `sites`, `equipements`, `liaisons`, `incidents`, `suivis`.
   - `incidents.reference` est unique (format `INC-2026-00042`) ;
   - choisir et **justifier** le comportement de suppression de chaque clé étrangère.
3. Créer les enums dans `app/Enums` : `Severite`, `StatutIncident`, `Categorie`, `Role`, `TypeSuivi`, avec `label()` et, si utile, `color()` (couleur Bootstrap).
4. Créer les modèles et **toutes** les relations.
5. Écrire le `DatabaseSeeder` : 1 admin, 4 techniciens, 8 sites, 2-3 équipements par site, 7 liaisons, ~40 incidents répartis sur les 6 dernières semaines.
6. Faire les 6 exercices MySQL et garder les requêtes dans `docs/requetes-semaine-2.sql`.

Livrable

`php artisan migrate:fresh --seed` construit une base complète et cohérente, et le fichier de requêtes SQL est commité.

Auto-évaluation

- [ ] Chaque clé étrangère a un comportement de suppression que je sais justifier
- [ ] `statut`, `severite` et `debut_at` sont indexés
- [ ] `$incident->severite` renvoie un objet `Severite`, pas une chaîne
- [ ] Toutes les relations fonctionnent dans Tinker
- [ ] Mes données de démo sont réalistes (`bko-core-rtr01`, vraies villes…)
- [ ] Je sais écrire un `JOIN` en SQL sans regarder mes notes

Ressources

- Laravel — Migrations : laravel.com/docs/migrations
- Laravel — Eloquent, relations : laravel.com/docs/eloquent-relationships
- Laravel — Factories et seeders : laravel.com/docs/seeding
- PHP — Enums : php.net/manual/fr/language.enumerations.php
- MySQL — JOIN : dev.mysql.com/doc/refman/8.4/en/join.html

Pour aller plus loin

Générer la référence de l'incident automatiquement dans l'événement `created` du modèle (indice : `static::created(...)` dans `booted()`, puis `saveQuietly()`). Pourquoi ne peut-on pas la calculer **avant** l'insertion ?